

Basic C Without Tears

Guía Práctica con Soluciones – Tipo A – Guía 03

Fundamentos de C, traza de código, strings y matrices

Autor: Stack

1. **Fundamentos del Lenguaje (Teoría aplicada).** Suponga que un `char` ocupa 1 byte y un `short` ocupa 2 bytes. Justifique las respuestas.

- (a) Para estas declaraciones, indique el valor de `sizeof(clave)`, cuántos caracteres útiles contiene la cadena y cuántos bytes suman ambos arreglos. Distinga la capacidad reservada de la longitud del texto.

```
char clave[7] = "LUNA";  
short intentos[4];
```

Solución y Explicación:

`sizeof(clave)` vale **7**: mide el arreglo completo, aunque el texto solo tenga **4 caracteres útiles**. El terminador ocupa una posición adicional; quedan dos posiciones sin usar por ese texto. `intentos` ocupa $4 \times 2 = 8$ bytes. Ambos arreglos suman $7 + 8 = \mathbf{15}$ bytes.

- (b) Indique el valor entero y el valor lógico de la expresión siguiente. Explique el efecto de `!` y de la división entera. ¿Se consideraría `-3` verdadero o falso si se usara solo como condición de un `if`?

```
!(-3) || (6 / 4 > 1)
```

Solución y Explicación:

`-3` es distinto de cero, por lo que es verdadero como condición. Su negación `!(-3)` vale 0. La división entera `6 / 4` vale 1, y `1 > 1` es falso. El resultado es `0 || 0`: **0, falso**.

- (c) Indique qué imprime el fragmento. ¿Sería válido agregar `printf("%d", usados);` después de cerrar el bloque? Explique el alcance de cada variable.

```
1 int cupos = 6;  
2 {  
3     int usados = 2;  
4     cupos -= usados;  
5 }  
6 printf("%d\n", cupos);
```

Solución y Explicación:

Imprime 4 seguido de un salto de línea. `cupos` está en alcance dentro del bloque y se modifica: $6 - 2 = 4$. `usados` solo está en alcance dentro de ese bloque; intentar utilizarlo después produciría un error de compilación por identificador no declarado.

- 2. Análisis y Traza de Código (Matemáticas y Control de Flujo).** ¿Qué imprime exactamente este código en pantalla? Suponga que el fragmento está dentro de `main` y que se incluyó `stdio.h`. Sin ejecutarlo ni modificarlo, respete los espacios y saltos de línea. Registre el dígito extraído, el valor restante de `n` y la suma al terminar cada vuelta, incluso cuando no se imprime nada.

```
1 int n = 5832, suma = 0;
2 while (n > 0) {
3     int digito = n % 10;
4     n /= 10;
5     if (digito % 2 != 0) {
6         continue;
7     }
8     suma += digito;
9     printf("%d:%d\n", digito, suma);
10 }
11 printf("Resto=%d Total=%d\n", n, suma);
```

Solución y Explicación:

El módulo obtiene el último dígito; la división entera lo elimina de `n`. Si el dígito es impar, `continue` salta la suma y la impresión, pero no deshace la división ya realizada.

digito	n restante	suma	Impresión
2	583	2	2:2
3	58	2	Ninguna
8	5	10	8:10
5	0	10	Ninguna

Salida exacta:

```
2:2
8:10
Resto=0 Total=10
```

Cada línea, incluida la última, termina en un salto de línea.

- 3. Algoritmia con Strings (Pseudocódigo).** Un catálogo acepta una cadena solo si sus letras están en orden alfabético no decreciente: cada letra debe ser mayor o igual que la anterior. Escriba el pseudocódigo de una función que retorne VERDADERO o FALSO según esa regla, sin modificar la cadena. Suponga que la entrada contiene únicamente letras de 'a' a 'z' en ASCII. Recorra la cadena hasta '\0' y no utilice funciones de biblioteca ni ordenamiento. Las letras repetidas se permiten; la cadena vacía y una cadena de una sola letra se consideran válidas.

Texto	Resultado
"abbcz"	VERDADERO
"abac"	FALSO
"m"	VERDADERO
" "	VERDADERO

Solución y Explicación:

```
FUNCION esta_ordenada(texto)
  i <- 0
  ordenada <- VERDADERO
  MIENTRAS texto[i] != '\0' HACER
    SI i > 0 ENTONCES
      SI texto[i] < texto[i - 1] ENTONCES
        ordenada <- FALSO
      FIN_SI
    FIN_SI
    i <- i + 1
  FIN_MIENTRAS
  RETORNAR ordenada
FIN_FUNCION
```

La condición `i > 0` evita leer la posición `-1`. La bandera queda falsa si aparece alguna disminución y no se restablece en comparaciones posteriores. Las letras iguales se aceptan. Para "abac", la segunda 'a' es menor que la 'b' anterior.

Una cadena vacía no entra al ciclo y una de una letra no compara vecinos: ambas retornan verdadero. El recorrido llega al terminador sin modificarlo.

4. **Algoritmia con Matrices/Arreglos (Pseudocódigo).** Escriba pseudocódigo que copie una matriz **origen** en otra matriz **espejo**, invirtiendo el orden de las columnas de cada fila. La primera columna debe pasar a ser la última, la segunda la penúltima, y así sucesivamente; las filas conservan su posición. Ambas matrices tienen **filas** por **columnas** posiciones, con dimensiones positivas y espacio ya reservado. Use bucles anidados e índices **i** y **j** desde cero, sin modificar **origen**. Muestre la matriz resultante para:

$$\text{origen} = \begin{bmatrix} 2 & 7 & 1 \\ 9 & 4 & 6 \end{bmatrix}$$

La solución debe funcionar también si hay una sola columna.

Solución y Explicación:

```
PARA i <- 0 HASTA filas - 1 HACER
  PARA j <- 0 HASTA columnas - 1 HACER
    espejo[i][j] <- origen[i][columnas - 1 - j]
  FIN_PARA
FIN_PARA
```

El índice de fila se conserva y el de lectura recorre las columnas desde la última hasta la primera. **origen** no se modifica. Con una sola columna, $1 - 1 - 0 = 0$, por lo que cada elemento se copia a la misma posición.

$$\text{espejo} = \begin{bmatrix} 1 & 7 & 2 \\ 6 & 4 & 9 \end{bmatrix}$$